

Loan Compliance System

Getting Started Guide — for non-developers, running the app in Visual Studio Code

This guide walks you through running the Loan Compliance System demo on your own computer, step by step, with no assumed coding experience. It should take about 10–15 minutes the first time. Once you're happy with it, you can hand the whole project folder to your developers for the real integration work (see the last section).

Before you start: what you need

- The project folder you were given, containing at least: `app.py`, `requirements.txt`, `README.md`, and a `.streamlit` folder.
- A Windows, Mac, or Linux computer with an internet connection (only needed for the one-time setup).
- About 15 minutes, and permission to install two pieces of free software if they aren't already on your computer: Python and Visual Studio Code.

Note: The `.streamlit` folder starts with a dot, which makes it a "hidden" folder on some computers - Visual Studio Code will still show and use it normally, so you don't need to do anything special with it. Just keep it in the same folder as `app.py`.

Part 1 — Install Python (skip if you already have it)

Python is the programming language this app is written in. Visual Studio Code does not include it automatically, so it needs to be installed once.

1. Open your web browser and go to the link below.

Download page: <https://www.python.org/downloads/>

2. Click the yellow "Download Python" button for your operating system.
3. Run the installer. On Windows, make sure you tick the box that says "Add Python to PATH" before clicking Install — this step is easy to miss and important.
4. Finish the installer using the default options.

Note: If you already have Python installed, you can skip this whole part.

Part 2 — Install Visual Studio Code (skip if you already have it)

1. Go to the link below and click "Download".

Download page: <https://code.visualstudio.com/>

2. Run the installer using the default options.
3. Open Visual Studio Code once it's installed. In the left-hand sidebar, click the square "Extensions" icon and search for "Python". Install the official extension published by Microsoft.

Part 3 — Open the project folder

1. In Visual Studio Code, go to File → Open Folder... and select the folder that contains `app.py`, `requirements.txt`, and `README.md`.

2. Open a terminal inside Visual Studio Code: go to Terminal → New Terminal in the top menu. A command-line panel will appear at the bottom of the window — this is where you'll type the commands in the next steps.

Part 4 — Install the required packages and run the app

Everything below is typed into that terminal panel at the bottom of Visual Studio Code, one line at a time, pressing Enter after each line.

1. (Optional but recommended) Create an isolated environment for this project, so it doesn't interfere with anything else on your computer:

```
python -m venv venv
```

Then activate it. On Windows:

```
venv\Scripts\activate
```

On Mac or Linux:

```
source venv/bin/activate
```

Note: You'll know it worked if you see "(venv)" appear at the start of the line in your terminal. You'll need to repeat this "activate" line (only this one) every time you reopen the project.

2. Install the packages the app depends on:

```
pip install -r requirements.txt
```

This downloads a handful of libraries (Streamlit, pandas, plotly, etc.) — it can take a minute or two the first time.

3. Run the app:

```
streamlit run app.py
```

4. Your web browser should open automatically to the app (usually at <http://localhost:8501>). If it doesn't open by itself, copy that address from the terminal into your browser manually.
5. To stop the app, go back to the terminal panel and press Ctrl + C (Cmd + C on some Mac keyboards).

Part 5 — Using the app

- Review Workdesk: a table of loans that failed one or more compliance rules, with search, filters, checkboxes for bulk actions, and a per-row Actions dropdown to mark each one Remediated, Ignored, or Removed.
- Compliance & Portfolio Report: summary cards and two charts showing the overall health of the portfolio.
- OPA Rego Playground / Asset Value Predictor: illustrative demos — clearly marked in the app as not production tools.
- Reset 100k (top-right): generates a brand-new random demo portfolio if you want to start over.
- Import CSV / Export Audit CSV (top-right): swap in your own data, or save your current portfolio to a file.

Part 6 — Using your own data instead of the demo data

By default, the app makes up a realistic-looking portfolio of 100,000 loans so there's something to look at immediately. When you're ready to try your own data, there are two easy ways to do it without any coding:

Option A — Import a CSV while the app is running

1. Click "Import CSV" in the top-right of the app.
2. Choose a CSV file from your computer. It needs the same column names as the file you get from clicking "Export Audit CSV" first — exporting once and opening that file in Excel is the easiest way to see exactly what's expected.

Option B — Point the app at your CSV every time it starts

Open `app.py` in Visual Studio Code, find the line that says `DATA_SOURCE_MODE = "demo"` near the top of the file, and change it to:

```
DATA_SOURCE_MODE = "csv"
CSV_DATA_PATH = "data/my_loan_portfolio.csv"
```

Then save the file and run `"streamlit run app.py"` again.

Note: Anything more advanced than a CSV — pulling data out of PDFs, using a vector database, or asking an AI model to extract fields from free text — is exactly what the next section is for.

Part 7 — Handing this off to your developers

`app.py` is already organised and commented for a developer to take over from here. Specifically, point them to the section clearly marked "DATA SOURCE ADAPTERS" inside `app.py`. It contains four functions, each with a docstring explaining what it should do and an example to start from:

- `load_data_from_csv(...)` — already fully working; loads a CSV into the app.
- `load_data_from_pdf_vectors(...)` — a placeholder for extracting structured loan data out of PDF documents via a vector database.
- `load_data_from_llm_extraction(...)` — a placeholder for using an LLM (e.g. Anthropic Claude or OpenAI) to pull structured fields out of free text.
- `load_data_from_custom_source(...)` — the single place to combine any of the above (or your own database/API call) into one final data source.

Every one of those functions must return the same table shape (documented directly above them in `app.py`, under "REQUIRED OUTPUT SCHEMA") so the rest of the app keeps working unchanged no matter where the data comes from.

Also worth mentioning to your developers: this file has already been formatted with "black" and checked with "flake8" (both are standard, free Python code-quality tools), so it should drop straight into a normal development workflow.

Troubleshooting

- "python is not recognised" / "command not found": Python isn't installed, or wasn't added to PATH — repeat Part 1 and make sure to tick the PATH box on Windows.
- "pip is not recognised": same cause as above — reinstall Python and tick the PATH option, then restart Visual Studio Code.
- "zsh: command not found: streamlit" (common on Mac) or "streamlit is not recognised" (Windows): pip installed everything correctly, but the shortcut command "streamlit" isn't on your terminal's list of known commands yet. Two fixes, in order of preference:
 - a) Run the app with `"python3 -m streamlit run app.py"` instead of `"streamlit run app.py"` — this works every time, regardless of PATH.

- b) Or go back to Part 4 and make sure you created AND activated the virtual environment (you should see "(venv)" at the start of your terminal line) before running pip install — this fixes it more permanently.
- The browser opens but the page is blank or shows an error: check the terminal — Streamlit prints the exact error there, usually a missing package (rerun "pip install -r requirements.txt") or a typo in app.py if it's been edited.
- "Port 8501 is already in use": an earlier copy of the app is still running somewhere. Close that terminal, or run "streamlit run app.py --server.port 8502" to use a different port.
- Nothing seems to have installed: make sure you see "(venv)" at the start of your terminal line before running pip install, if you created a virtual environment in Part 4.
- "TypeError: ...() got an unexpected keyword argument ..." mentioning a Streamlit function: this means your installed Streamlit version is older than whatever feature the code is using. app.py is written to avoid this for its own features (verified to run on Streamlit versions as old as 1.45, including on Python 3.9, where Streamlit itself caps out at 1.50). If you see this error anyway - e.g. after an update from your developers - run "pip install --upgrade streamlit" (inside your activated virtual environment) and try again.

Quick reference

```
python -m venv venv
venv\Scripts\activate      (Windows)
source venv/bin/activate   (Mac/Linux)
pip install -r requirements.txt
streamlit run app.py
python3 -m streamlit run app.py  (use this if "streamlit" isn't found)
```